



UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI
INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA
E SISTEMISTICA

DIMES

***Realizzazione di una applicazione distribuita per la
sottomissione di questionari agli utenti***

Studente

Marco Ziccardi

Matricola 256938

Appello del 13/07/2024

Sommario

1. Introduzione – scopi e obiettivi del progetto	1
2. Descrizione del Sistema.....	2
2. Analisi dei Requisiti	3
2.1 <i>Requisiti Funzionali</i>	3
2.2 <i>Requisiti non funzionali.....</i>	4
2.3 <i>Caratteristiche supplementari.....</i>	4
3. Architettura del Sistema.....	5
3.1 <i>Schema architetturale.....</i>	9
3.2 <i>Use Case Diagram</i>	10
4. Implementazione	11
4.1 <i>Sviluppo del Back-end.....</i>	11
4.2 <i>Sviluppo del Front-end.....</i>	14
5. Vantaggi nell'uso dei servizi Cloud	14
6. Conclusioni.....	15

1. Introduzione – scopi e obiettivi del progetto

La presente relazione ha lo scopo di proporre la progettazione e lo sviluppo di un'applicazione distribuita per la somministrazione di questionari agli utenti.

Il progetto presentato ha l'obiettivo di sviluppare una piattaforma che permetta agli utenti di creare, somministrare e analizzare questionari. Le principali funzionalità dell'applicazione includono:

- *Caricamento dei questionari*: gli utenti possono creare e caricare questionari, composti da una o più domande a risposta multipla, con la possibilità di includere immagini.
- *Invito agli utenti*: gli utenti possono invitare altri partecipanti a rispondere ai questionari attraverso e-mail.
- *Risposta ai questionari*: ogni utente può rispondere a tutti i questionari disponibili sulla piattaforma una sola volta entro la scadenza della campagna di sottomissione fornita.
- *Visualizzazione dei risultati*: i risultati parziali e finali dei questionari possono essere visualizzati in qualsiasi momento.

Il progetto, attraverso le risorse cloud di Amazon AWS, mira a creare un'applicazione scalabile, sicura e performante, garantendo un'esperienza ottimale per l'utente.

Questa relazione andrà a descrivere gli obiettivi del progetto, l'analisi dei requisiti funzionali e non funzionali, l'architettura del sistema, le tecnologie utilizzate e il processo di implementazione. Inoltre, verranno fornite informazioni sui test eseguiti per garantire la qualità del sistema e verrà descritta una demo funzionante dell'applicazione.

2. Descrizione del Sistema

Il presente sistema, andando più nel dettaglio, realizza un'applicazione web distribuita per la somministrazione di questionari agli utenti.

Ogni utente, dunque, dopo essersi registrato ed aver effettuato il login ha la possibilità di interagire con il sistema creato, potendo usufruire delle funzionalità proprie del sistema stesso. In particolare, le funzionalità appena citate sono le seguenti:

- *Lista questionari*, con la quale l'utente ha la possibilità di prendere visione nel dettaglio di tutti i questionari che gli utenti registrati hanno creato. Inoltre, tale schermata è accessibile a tutti gli utenti, anche coloro che non sono registrati. Tale funzionalità è stata creata al fine di dare la possibilità a tutti gli utenti di visionare questionari appartenenti alla loro area di interesse, così da invogliare l'utente stesso a registrarsi per poter compilare i questionari e aumentare la percentuale di partecipazione.
- *Crea questionario*, in cui è possibile, da parte di un utente registrato, dare vita ad un questionario che riguardi vari campi di interesse. Il questionario sarà composto da un *titolo*, da una *descrizione* specifica, da una *durata* in giorni in cui è possibile compilare un questionario prima della scadenza e da un elenco di *domande a risposta multipla* in cui si può specificare: il *testo della domanda*, il *numero di risposte* che si vuole inserire, i *testi delle risposte* ed infine sarà anche possibile il *caricamento di una immagine* riferita alla domanda stessa.

Una volta creato il questionario, l'utente può accedere a una schermata di notifica in cui ha la possibilità di inviare, a uno o più utenti da lui selezionati, via e-mail un invito ad effettuare l'accesso per poter compilare il nuovo questionario. Il contenuto dell'e-mail è predefinito, ma qualora l'utente volesse può anche modificarlo.

- *Compila questionario*, qui l'utente ha la possibilità di visualizzare tutti i questionari, sia quelli disponibili per la compilazione (che non sono scaduti) sia quelli non disponibili per la compilazione (poiché scaduti). In particolare quest'ultimi, oltre a presentare nella descrizione il termine "SCADUTO", non saranno "cliccabili" per poter proseguire alla compilazione. Per quanto concerne invece i questionari compilabili, nel momento in cui l'utente andrà a cliccare sul questionario da lui scelto gli comparirà una schermata in cui saranno visibili tutte le domande appartenenti al questionario. Laddove nella domanda siano state caricate delle immagini, l'utente avrà la possibilità di effettuare il download della immagine stessa. L'utente ha la possibilità di poter rispondere a uno o più quesiti presentati e a sua volta a una o più risposte. Se l'utente ha già provveduto alla compilazione di un questionario, quest'ultimo non sarà più visibile tra i questionari compilabili.
- *Statistiche*, tale sezione è invece dedicata alla visualizzazione per l'utente dei risultati parziali e/o finali di ogni singolo questionario. Nel momento in cui la campagna di somministrazione di un questionario non è ancora terminata sarà possibile visualizzare i risultati parziali. Al contrario, al termine della campagna saranno visualizzabili i

risultati finali. Per poter visualizzare i risultati di un questionario bisognerà naturalmente cliccare sul questionario stesso; una volta effettuato ciò l'utente si troverà di fronte ad una schermata in cui ci sarà il resoconto di ogni domanda. A fianco ad ogni risposta verrà mostrato il numero di voti totalizzati per quella determinata risposta. Tale sezione è visualizzabile da tutti gli utenti anche se non registrati nel sistema.

Il sistema è stato realizzato mediante l'utilizzo dei servizi Amazon AWS (quali storage, autenticazione, invio di e-mail e database), dai quali il sistema è dunque totalmente dipendente.

2. Analisi dei Requisiti

Per delineare chiaramente le caratteristiche che il sistema di interesse deve avere, è indispensabile esaminare tutti i requisiti funzionali e non funzionali ad esso correlati.

2.1 Requisiti Funzionali

I requisiti funzionali sono descritti nella tabella seguente:

Nome Requisito	Descrizione
Registrazione utente	Il sistema deve permettere agli utenti di registrarsi inserendo la coppia username e password
Login utente	Il sistema deve permettere agli utenti registrati di poter accedere al sistema
Logout utente	Il sistema deve permettere agli utenti che hanno effettuato l'accesso di poter uscire dal sistema
Caricamento dei questionari	Il sistema deve permettere agli utenti registrati di poter creare e caricare questionari
Struttura del questionario	Ogni questionario può includere una o più domande a risposta multipla
Caricamento di immagini	Deve essere possibile caricare immagini per ogni domanda
Invito agli utenti	Il sistema deve permettere agli utenti registrati, dopo aver creato il questionario, di notificare tramite e-mail il nuovo questionario ad altri utenti
Compilazione di un questionario	Il sistema deve permettere agli utenti che hanno effettuato l'accesso di compilare una sola volta un questionario
Download di immagini	Il sistema deve permettere all'utente di scaricare, quindi visualizzare, le immagini

Visualizzazione dei Risultati	Il sistema deve permettere agli utenti di avere una visualizzazione in tempo reale dei risultati parziali e/o finali dei questionari
-------------------------------	--

2.2 Requisiti non funzionali

I requisiti non funzionali sono descritti nella tabella seguente:

Nome Requisito	Descrizione
Usabilità	Il sistema fornisce una interfaccia utente intuitiva e facile da navigare e usare
Compatibilità del browser	Il sistema deve essere compatibile con i diversi browser disponibili in commercio
Sicurezza	Il sistema deve garantire un accesso controllato attraverso l'uso di credenziali
Scalabilità	Il sistema deve essere progettato per scalare al fine di gestire un numero crescente di utenti e questionari senza compromettere le prestazioni, utilizzando servizi di cloud AWS
Manutenibilità	Il codice sorgente e l'architettura del sistema sono stati progettati per facilitare la manutenzione continua e l'aggiornamento del software
Affidabilità	Il sistema garantisce una memorizzazione affidabile dei dati, senza che vi siano perdite, tramite i servizi di cloud AWS

2.3 Caratteristiche supplementari

Oltre ai requisiti fondamentali precedentemente illustrati, è stata identificata una caratteristica supplementare che, sebbene non esplicitamente richiesta, può potenziare l'efficacia del sistema.

La caratteristica supplementare implementata è descritta nella seguente tabella:

Nome Requisito	Descrizione
Scadenza questionario	Il sistema deve consentire la compilazione di un questionario solamente entro i limiti di scadenza stabiliti durante la creazione del questionario stesso.

3. Architettura del Sistema

È necessario ora fornire una descrizione dettagliata delle singole componenti che hanno permesso di soddisfare i requisiti funzionali e non, descritti in precedenza, che hanno consentito la realizzazione dell'applicativo web. Di seguito è presentata una panoramica delle componenti principali del sistema e delle loro interazioni.

Componenti principali dell'architettura:

1. Interfaccia Utente (Front-end)

- Realizzata attraverso il framework flutter per garantire un'esperienza utente interattiva e reattiva.
- Ha permesso di realizzare una serie di funzionalità, tra cui: effettuare il login, effettuare la registrazione, effettuare il logout, possibilità di creazione e compilazione dei questionari, selezione degli utenti a cui inviare l'e-mail, caricamento e download di immagini e visualizzazione dei risultati.

In particolare, il front-end, dopo essere stato sviluppato correttamente, è stato caricato all'interno del servizio AWS CloudFront. Ciò ha permesso di distribuire i contenuti statici e dinamici dell'applicativo tramite una rete globale di edge location, riducendo la latenza e migliorando le prestazioni di consegna.

2. Server Applicativo (Back-end)

- Realizzato tramite il framework spring.io che ha permesso di gestire la logica di business e le interazioni con il database e altri servizi AWS.
- Ha permesso di realizzare una serie di funzionalità, tra cui: servizio inerente alla registrazione, al login e al logout tramite l'utilizzo del servizio AWS Cognito; servizio inerente al salvataggio dei questionari, delle compilazioni e dei risultati parziali e totali tramite il servizio AWS DynamoDB; servizio inerente al caricamento e download delle immagini tramite il servizio AWS Simple Storage Service; servizio inerente all'invio degli inviti tramite e-mail eseguito attraverso il servizio AWS Simple Email Service.

In particolare, il back-end, dopo essere stato sviluppato correttamente, è stato caricato all'interno del servizio AWS Elastic Beanstalk. Si è deciso di utilizzare tale servizio poiché esso consente una semplicità di deployment, una gestione automatica dell'infrastruttura, una scalabilità automatica e un'integrazione con altri servizi AWS (come per il "Monitoraggio e Logging"). Tutto ciò ha permesso di realizzare un'applicazione robusta, scalabile e performante.

3. Database NoSQL DynamoDB

- Esso è stato utilizzato per rendere permanenti i dati inerenti all'applicazione web, in particolare i dati relativi ai questionari, ai risultati, e alle varie compilazioni.

- Nello specifico, gli schemi adottati per la gestione dei dati hanno un formato JSON e sono strutturati nel seguente modo:

1. Salvataggio questionario

```
{
  "id_questionario": "...",
  "nome_autore": "...",
  "data_ora": "AAAA-MM-GG OO:MM",
  "titolo": "...",
  "descrizione": "...",
  "durata": 0,
  "domande": [
    {
      "domanda": "...",
      "risposte": [
        "...",
        "...",
        "...",
        "...",
        ...
      ],
      "url_immagine": "..."
    },
    ...
  ]
}
```

Analizzando tale struttura, ritroviamo i seguenti campi principali:

1. id_questionario (chiave di partizione), un identificatore unico per il questionario.
2. nome_autore (chiave di ordinamento), il nome dell'autore del questionario.
3. data_ora, la data e l'ora in cui è stato creato o pubblicato il questionario, nel formato "AAAA-MM-GG OO:MM".
4. titolo, il titolo del questionario.
5. descrizione, una descrizione del questionario.
6. durata, indica i giorni di tempo disponibili per la compilazione del questionario.
7. domande, un array che contiene i quesiti del questionario. Ogni domanda ha i seguenti campi: domanda (il testo della domanda), risposte (un array di possibili risposte alla domanda) e url immagine (un url opzionale che punta a un'immagine correlata alla domanda).

La struttura consente di memorizzare informazioni dettagliate e configurabili riguardo a un questionario e le sue domande, rendendo facile la gestione e la visualizzazione dei dati per applicazioni come sondaggi o esami online.

2. Risultati questionario

```
{
  "id_risultato": "...",
  "nome_autore": "...",
  "data_ora": "AAAA-MM-GG OO:MM",
  "descrizione": "...",
  "numeroRisposte": 0,
  "durata": 0,
  "titolo": "...",
  "resocontoDomande": [
    {
      "domanda": "...",
      "url_immagine": "...",
      "numeroRisposta": [
        {
          "numero": 0,
          "risposta": "..."
        },
        ...
      ],
      ...
    },
    ...
  ],
  ...
}
```

L'immagine raffigurata rappresenta il resoconto di un questionario completato, avente i seguenti campi principali:

1. id_risultato (chiave di partizione), un identificatore unico per il risultato analogo all'id del questionario.
2. nome_autore, il nome dell'autore del questionario.
3. data_ora, la data e l'ora in cui il risultato è stato registrato, nel formato "AAAA-MM-GG OO:MM".
4. descrizione, una breve descrizione del questionario.
5. numeroRisposte, il numero totale di risposte fornite nel questionario.
6. durata, la durata totale impiegata per poter completare il questionario, espressa in giorni.
7. titolo, il titolo del questionario o del resoconto.

8. `resocontoDomande`, un array contenente il resoconto delle domande del questionario. Ogni elemento dell'array ha i seguenti campi: `domanda` (il testo della domanda), `url_immagine` (un URL opzionale che punta a un'immagine associata alla domanda) e `numeroRisposta` (un array di risposte fornite per quella domanda).

Ogni `numeroRisposta` ha i seguenti campi: `numero` (quanti voti ha ricevuto la specifica risposta) e `risposta` (il testo della risposta data).

Questa struttura JSON consente di memorizzare dettagli completi su come è stato completato un questionario, inclusi i tempi, le risposte fornite e le immagini correlate alle domande.

3. Compilazioni questionari

```
{
  "id_questionario": "...",
  "nome_utente": "...",
  "data_ora": "AAAA-MM-GG OO:MM",
  "risposte": [
    {
      "domanda": "...",
      "scelte": [
        "...",
        "...",
        "..."
      ]
    },
    ...
  ],
  ...
}
```

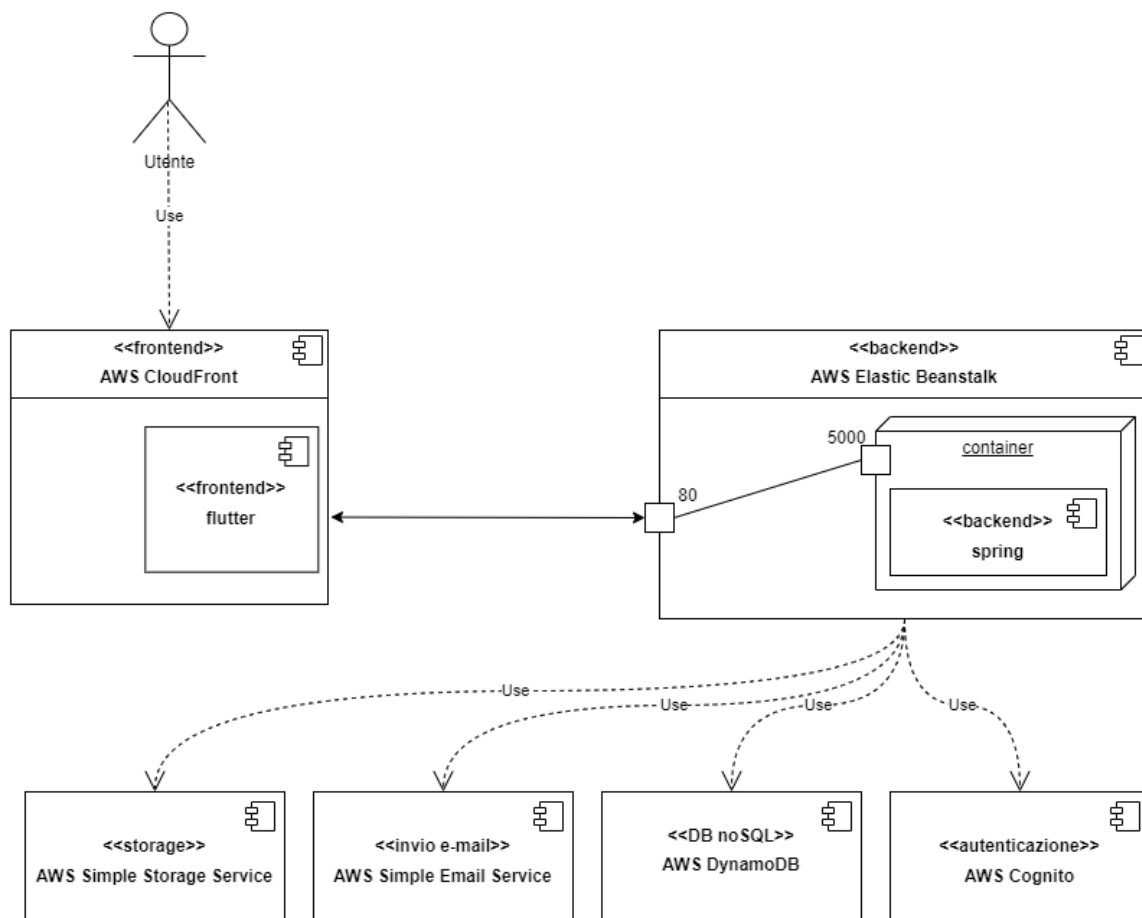
L'immagine mostra una struttura JSON che rappresenta le risposte di un utente a un questionario. I campi della struttura sono i seguenti:

1. `id_questionario` (chiave di ordinamento), identifica il questionario compilato.
2. `nome_utente` (chiave di partizione), il nome dell'utente che ha fornito le risposte.
3. `data_ora`, la data e l'ora in cui le risposte sono state registrate, nel formato "AAAA-MM-GG OO:MM".
4. `risposte`, un array che contiene le risposte fornite per ciascuna domanda del questionario. Ogni elemento dell'array ha i seguenti campi: `domanda` (il testo della

domanda alla quale l'utente ha risposto) e scelte (un array di risposte selezionate dall'utente per quella domanda).

Questa struttura permette di conservare le risposte date da un utente a un questionario specifico, compresa l'identificazione del questionario, il nome dell'utente e le risposte selezionate per ogni domanda.

3.1 Schema architetturale



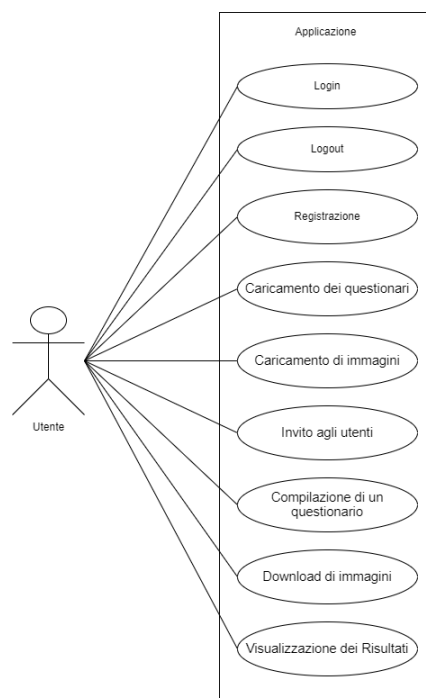
L'immagine sopra fornita rappresenta l'architettura del sistema distribuito per la realizzazione dell'applicativo web. I vari componenti hanno ognuno uno specifico ruolo come sopra illustrato. Andiamo a vedere nel dettaglio come questi interagiscono tra di loro:

- Interazione *Utente* - *AWS CloudFront*, grazie alla quale l'utente accede all'applicazione, dunque al front-end, tramite il servizio CloudFront, che distribuisce i file statici e dinamici dell'applicazione Flutter.

- Interazione *AWS CloudFront - Flutter*, che provvede a servire il contenuto sviluppato per mezzo di flutter.
- Interazione *AWS CloudFront - AWS Elastic Beanstalk*, l'applicazione Flutter comunica con il back-end gestito da Elastic Beanstalk. Le richieste dell'utente vengono inviate al server back-end che le esegue e restituisce il risultato.
- Interazione *AWS Elastic Beanstalk – Spring*, Elastic Beanstalk esegue in maniera automatica l'applicazione back-end in Spring all'interno di un container, gestendo il deployment, la scalabilità e numerosi altri servizi. In particolare, le varie richieste e risposte vengono mappate dalla porta interna del servizio spring alla porta esposta all'esterno di Elastic Beanstalk.
- Interazione *AWS Elastic Beanstalk - AWS Simple E-mail Service*, attraverso le API per inviare e-mail.
- Interazione *AWS Elastic Beanstalk - AWS DynamoDB*, attraverso le API Spring interagisce con DynamoDB per memorizzare e recuperare i dati no SQL.
- Interazione *AWS Elastic Beanstalk - AWS Cognito*, attraverso le API per autenticare gli utenti, gestire le sessioni di login e le credenziali di accesso.

3.2 Use Case Diagram

Tale schema rappresenta le funzionalità derivate dall'iterazione tra l'utente e il sistema proposto:



4. Implementazione

4.1 Sviluppo del Back-end

Come precedentemente anticipato, lo sviluppo del back-end è stato effettuato tramite il framework Spring Boot che utilizza il linguaggio di programmazione Java (in particolare la versione 17).

Tale framework ha permesso di effettuare uno sviluppo stratificato in cui le componenti principali sono:

- Package di *Configurazioni* (*AWSConfig*, *EncodingConfiguration* e *OAuth2LoginSecurityConfig*), che hanno permesso di effettuare l'interfacciamento con i servizi del cloud AWS tramite delle chiavi di accesso e hanno anche permesso di configurare le politiche di sicurezza del back-end.
- Package dei *Controller* (*CompilazioneController*, *EmailController*, *ImmaginiController*, *QuestionarioController*, *RisultatiController* e *UtenteController*), grazie ai quali il back-end può essere contattato dall'esterno e di conseguenza fornisce i vari servizi intrinseci.
- Package dei *Servizi* (*CognitoService*, *CompilazioneService*, *EmailService*, *QuestionarioService*, *RisultatiService* e *StorageService*), che hanno messo a disposizione tutti i metodi che hanno consentito l'implementazione della logica di business della applicazione ed inoltre tutti quei metodi utili per l'interfacciamento con le API dei servizi cloud.
- Package delle *Entity* (*Compilazione*, *Domanda*, *LoginRequest*, *NumeroRisposta*, *Questionario*, *ResocontoDomande*, *Risposta*, *Risultati* e *Utente*), che sono serviti per rappresentare le entità presenti nel database.

Tra i quattro package sopra citati, quello che merita un approfondimento maggiore è l'insieme di classi appartenente ai servizi.

La classe *CognitoService* è un servizio Java che interagisce con AWS Cognito per la gestione degli utenti, fornendo funzionalità di registrazione, autenticazione e recupero degli utenti. Questa classe utilizza il client AWS Cognito Identity Provider (*AWSCognitoIdentityProvider*) per effettuare le operazioni richieste:

- Metodo "login", autentica un utente utilizzando il flusso di autenticazione *USER_PASSWORD_AUTH* di Cognito. Se l'autenticazione ha successo, restituisce un oggetto JSON con i token di accesso (*AccessToken*, *IdToken*, *RefreshToken*).

- Metodo “registerUser”, registra un nuovo utente con le informazioni fornite (username, e-mail, password). Se la registrazione ha successo, restituisce true; altrimenti, false.
- Metodo “getTuttiUtenti”, recupera tutti gli utenti del pool di utenti specificato (userPoolId). Restituisce una lista di oggetti UserType, che rappresenta gli utenti registrati.

La classe *QuestionarioService* è un servizio Java che gestisce operazioni CRUD (Create, Read, Update, Delete) sulle entità Questionario, utilizzando DynamoDB come database. Inoltre, crea e aggiorna un'entità Risultati associata al Questionario.

Analizziamo ora il funzionamento della classe, considerando il metodo più importante:

- Metodo “save”, salva un nuovo questionario e crea un record Risultati associato. Per ogni domanda nel questionario, crea un ResocontoDomande con il conteggio delle risposte inizializzato a zero e lo salva nel servizio RisultatiService.

La classe *CompilazioneService* è un servizio Java che gestisce operazioni CRUD (Create, Read, Update, Delete) sulla entità Compilazione, utilizzando DynamoDB come database.

I metodi più importanti sono:

- Metodo “save”, salva una nuova compilazione ed aggiorna contemporaneamente il conteggio delle risposte nel corrispondente record Risultati. Per ogni risposta della compilazione, incrementa il conteggio delle risposte nel resoconto delle domande.
- Metodo “getQuestionariDisponibili”, restituisce una lista di questionari disponibili per un determinato utente, escludendo quelli che l'utente ha già compilato in modo tale da effettuare un'unica compilazione per i questionari disponibili.

La classe *EmailService* è un servizio Java che utilizza le API AWS Simple E-mail Service (SES) per inviare e-mail singole e multiple. Analizziamo il funzionamento della classe:

- Metodo “singoloInvio”, invia un'e-mail singola all'indirizzo specificato (to). Crea un oggetto SendEmailRequest che contiene la destinazione, il soggetto e il corpo del messaggio, e lo invia utilizzando l'API sendEmail().
- Metodo “inviiMultipli”, invia e-mail a più destinatari iterando attraverso un array di indirizzi e-mail (to). Crea e invia un SendEmailRequest per ogni indirizzo e-mail. Utilizza Thread.sleep(2000) per aggiungere una pausa di 2 secondi tra gli invii per evitare problemi di rate limiting.

La classe *RisultatiService* è un servizio che gestisce le operazioni CRUD (Create, Read, Update, Delete) sull'entità Risultati, utilizzando DynamoDB come database. La classe sfrutta DynamoDBMapper per interagire con DynamoDB.

La classe *StorageService* è un servizio Spring che gestisce le operazioni di caricamento, download e cancellazione di file in un bucket Amazon S3. Utilizza le API AWS SDK per Java per interagire con S3.

Di seguito una spiegazione dettagliata del funzionamento della classe:

- Metodo “*uploadFile*” gestisce il caricamento di un file su Amazon S3. In particolare, converte il *MultipartFile* in un oggetto *File* utilizzando il metodo *convertMultiPartFileToFile*; genera un nome univoco per il file basato sul timestamp corrente e sul nome originale del file. Infine, utilizzando il metodo *.putObject()*, carica il file nel bucket S3 specificato. Restituisce l’URI della risorsa all’interno di S3.
- Il metodo “*downloadFileNome*” gestisce il download di un file da Amazon S3 utilizzando il nome del file. Impiega il metodo *getObject()* per ottenere l’oggetto dal bucket S3 usando il nome del file. Nel caso di corretto scaricamento ritorna il contenuto dell’immagine sotto forma di byte.
- Il metodo “*downloadFileURI*”, a differenza del precedente, gestisce il download di un file da Amazon S3 utilizzando un URI S3.
- Il metodo “*deleteFile*” gestisce la cancellazione di un file dal bucket Amazon S3.

Per far fronte alla funzionalità intrinseca di AWS Cognito, in cui per poter registrarsi alla applicazione è necessario confermare tramite e-mail la veridicità dell’indirizzo, ho utilizzato il servizio AWS Lambda, grazie al quale è stato possibile rendere valide anche e-mail apocrife.

La funzione lambda, che mi ha consentito quanto descritto, è la seguente:

```
const handler = async (event) => {  
  // Confirm the user  
  event.response.autoConfirmUser = true;  
  // Set the email as verified if it is in the request  
  if (event.request.userAttributes.hasOwnProperty("email")) {  
    event.response.autoVerifyEmail = true;  
  }  
  
  return event;  
};  
  
export { handler };
```

In particolare, questa funzione JavaScript interagisce con l'evento, proveniente da Amazon Cognito, di registrazione degli utenti, andando a confermare la verifica dell'e-mail e confermando anche l'utente.

4.2 Sviluppo del Front-end

Per la creazione del front-end, ossia la parte della applicazione con cui si interfaccia l'utente, è stato utilizzato il framework Flutter che si avvale del linguaggio di programmazione Dart.

Esso espone principalmente le seguenti funzionalità:

- Interfaccia grafica per l'interazione con l'utente.
- Metodi per la comunicazione con il back-end, per richiedere i servizi di quest'ultimo da parte dell'utente.
- Gestione dell'autenticazione dei clienti.

L'aspetto fondamentale di questa componente è la gestione dell'autenticazione dei client e la comunicazione sicura con il back-end, in modo da consentire l'invio di richieste contenenti le informazioni necessarie per riconoscere l'identità degli utenti. Per effettuare quanto descritto, il front-end comunica con il back-end per ottenere il token access. Una volta ottenuto lo si tiene in memoria per poterlo andare ad utilizzare successivamente all'interno dell'header delle varie richieste verso il back-end, che richiedono di essere autenticate. Il token access, quindi, serve per rappresentare l'utente nelle varie richieste.

5. Vantaggi nell'uso dei servizi Cloud

I vantaggi che sono emersi dall'utilizzo dei servizi cloud sono diversi. Mi ha permesso di esonerarmi dalla implementazione di diversi aspetti progettuali, consentendomi di concentrarmi maggiormente sullo sviluppo delle funzionalità dell'applicazione anziché sulla gestione dell'infrastruttura. I vantaggi principali sono:

1. *Scalabilità*, Elastic Beanstalk e DynamoDB garantiscono che il sistema possa scalare automaticamente per gestire variazioni importanti in termini di carico di lavoro e di volumi di dati senza avere una diminuzione di prestazioni.
2. *Prestazioni*, l'uso di CloudFront riduce la latenza e migliora le prestazioni della distribuzione dei contenuti statici e dinamici.
3. *Sicurezza*, AWS Cognito fornisce una gestione sicura delle credenziali degli utenti e dell'autenticazione.
4. *Affidabilità*, l'infrastruttura di AWS, insieme ai meccanismi di failover e replica di DynamoDB, garantisce alta disponibilità e affidabilità dei dati.

5. *Facilità di Gestione*, Elastic Beanstalk semplifica la gestione del deployment e dell'infrastruttura, automatizzando operazioni complesse come il provisioning di server, bilanciamento del carico e scaling automatico.

Questa architettura rappresenta un sistema ben integrato e altamente performante per la gestione e la somministrazione di questionari, sfruttando appieno i servizi cloud offerti da AWS.

6. Conclusioni

In conclusione, questo progetto si è concentrato sullo sviluppo di un'applicazione web per la gestione e la compilazione di questionari online, sfruttando pienamente le potenzialità dei servizi AWS Cloud. Attraverso un'attenta progettazione e implementazione delle componenti back-end e front-end, si è garantito che il sistema rispondesse efficacemente sia ai requisiti funzionali che a quelli non funzionali, come la affidabilità dei dati e la scalabilità dell'infrastruttura.

L'integrazione di tecnologie come Amazon DynamoDB per la gestione dei dati dei questionari, Amazon S3 per lo storage dei file multimediali associati, e AWS Lambda per la gestione di eventi ha permesso di creare una soluzione robusta e altamente scalabile. Questa architettura ha facilitato la gestione efficiente dei dati dei questionari e la risposta dinamica alle esigenze di carico di lavoro variabili.

Inoltre, sono stati adottati meccanismi avanzati di sicurezza forniti da AWS, come Amazon Cognito per l'autenticazione degli utenti e il controllo degli accessi, garantendo che le informazioni sensibili degli utenti fossero protette in modo adeguato.

Infine, il progetto è stato creato per essere estensibile e pronto per ulteriori miglioramenti. L'utilizzo di servizi cloud e architetture serverless permette di aggiungere facilmente nuove funzionalità e di gestire aggiornamenti senza interruzioni per gli utenti finali.

Il progetto è disponibile al seguente link: <http://frontendquestionario.s3-website.eu-north-1.amazonaws.com>